

Headless DMS - Architecture Guide

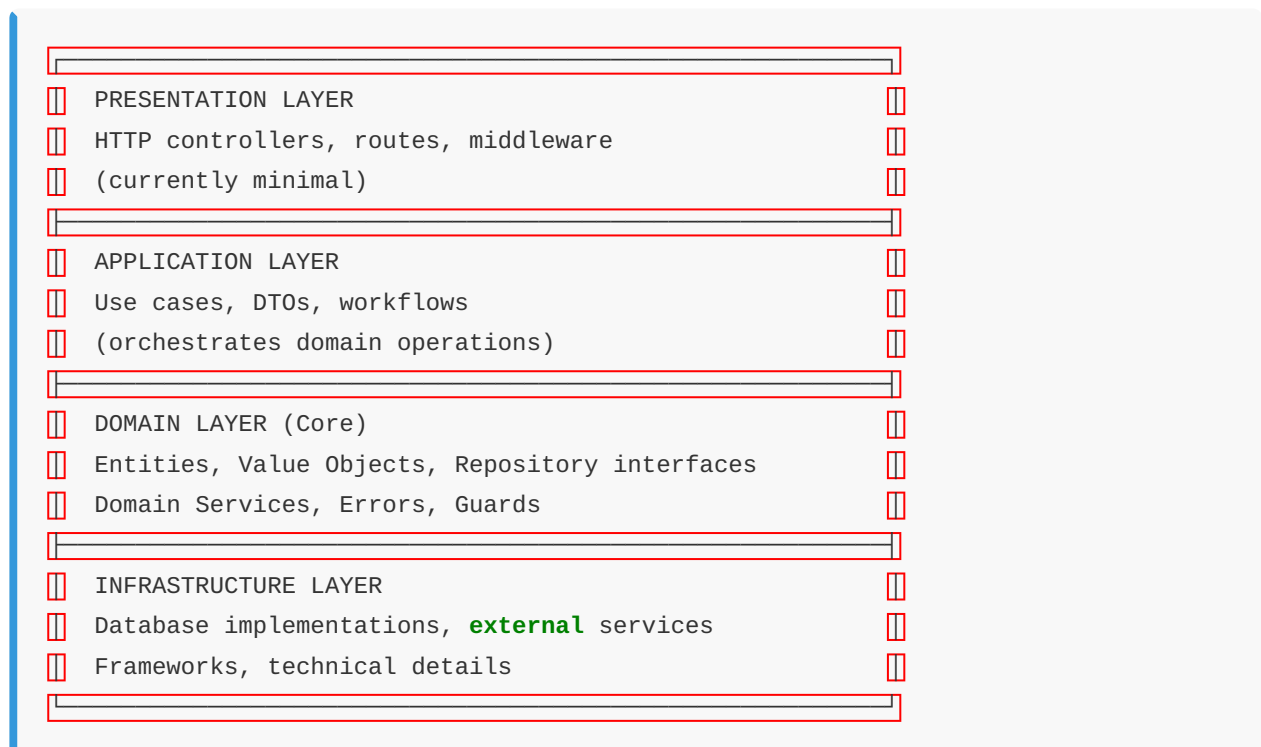
Table of Contents

1. [Layered Architecture Overview](#)
 2. [Clean Architecture Principles](#)
 3. [Factory Pattern](#)
 4. [Base Entity](#)
 5. [Repository Pattern](#)
 6. [Refined Types \(Branded Types\)](#)
 7. [Functional Error Handling](#)
 8. [Domain Error Hierarchy](#)
 9. [Dependency Injection](#)
 10. [Serialization & Rehydration](#)
-

1. Layered Architecture Overview

We follow **Clean Architecture** (also called Onion Architecture), which organizes code into concentric layers with **dependencies pointing inward** toward the domain core.

Architecture Diagram



Why This Architecture?

Benefit	Explanation
Separation of Concerns	Business logic lives in the domain, isolated from frameworks
Testability	Domain can be tested without database or HTTP server
Flexibility	Swap PostgreSQL for MongoDB without touching domain code
Maintainability	Changes in one layer don't cascade to others
Framework Independence	Domain doesn't depend on Express, Drizzle, or any external library

Directory Structure

```

src/
├── domain/                # Core business logic
│   ├── shared/           # Base classes (entity, repository, errors)
│   ├── user/             # User aggregate
│   ├── document/         # Document aggregate
│   ├── access-policy/    # Access control
│   ├── audit-log/        # Audit trail
│   ├── services/         # Domain services
│   ├── utils/            # Utilities
│   └── refined-types/    # Branded types
├── value-objects/         # Value objects (DateTime, etc.)
├── application/           # Use cases (currently minimal)
├── infra/                 # Infrastructure implementations
│   ├── db/               # Database models and schema
│   └── models/            # Drizzle ORM models
├── repositories/          # Repository implementations
├── presentation/         # HTTP layer
│   └── http/              # Controllers and routes

```

Path Mapping (tsconfig.json)

```

{
  "compilerOptions": {
    "paths": {
      "@domain/*": ["src/domain/*"],
      "@application/*": ["src/application/*"],
      "@infra/*": ["src/infra/*"],
      "@presentation/*": ["src/presentation/*"],
      "@shared/*": ["src/shared/*"]
    }
  }
}

```

2. Clean Architecture Principles

The Dependency Rule

Dependencies only point inward. The inner layers (Domain) know nothing about the outer layers (Infrastructure, Presentation).

Presentation  depends on  > Application  depends on  > Domain
Infrastructure  implements  > Domain (Repository interfaces)

Layer Responsibilities

Layer	Responsibility	Knows About	Doesn't Know About
Domain	Business logic, invariants	Value objects, entities, domain events	Database, HTTP, frameworks
Application	Orchestration, use cases	Domain, DTOs	Database details, HTTP
Infrastructure	Technical implementation	Application, Domain interfaces	Business rules
Presentation	HTTP handling, serialization	Application, DTOs	Domain entities, DB

Key Principle: Domain Never Imports Infrastructure

Correct:

```
// Infrastructure imports Domain
import { UserRepository } from "@domain/user/user.repository";

@injectable()
export class UserRepositoryImpl implements UserRepository {
  // Implementation
}
```

Incorrect:

```
// Domain should NEVER import Infrastructure
import { UserRepositoryImpl } from "@infra/repositories/user.repository.impl"; // ❌ Wrong!
```

3. Factory Pattern

Factories are **static methods that control object creation**. Every entity has two factory methods that serve different purposes.

The Two Factory Methods

3.1 `create()` - For New Entities

Generates ID and timestamps automatically. Used when creating new business objects.

File: `src/domain/user/user.entity.ts`

```
export class User extends BaseEntity<UserId> implements IUser {
  /**
   * Factory: creates a new User with a generated ID and timestamps.
   * Caller is responsible for hashing the password before passing it in.
   */
  static create(data: CreateEntity<IUser>): Result<User, Error> {
    const now = DateTime.now();
    return Result.Ok(
      new User({
        ...data,
        id: UserId.init(),           // Auto-generated UUID
        createdAt: now,             // Auto-generated timestamp
        updatedAt: now,
      })
    );
  }
}
```

Usage:

```

const result = User.create({
  email: Email.create("user@example.com").unwrap(),
  passwordHash: await bcrypt.hash(password, 10),
  role: UserRole.USER,
  displayName: "John Doe",
  isActive: true
});

if (result.isOk()) {
  const user = result.unwrap();
  // User has: id, createdAt, updatedAt automatically set
}

```

3.2 fromSerialized() - For Rehydration

Reconstructs entity from database or API format. Used when loading existing data.

File: `src/domain/user/user.entity.ts`

```

export class User extends BaseEntity<UserId> implements IUser {
  /**
   * Factory: rehydrates a User from its serialized (persistence) form.
   */
  static fromSerialized(raw: SerializedUser): User {
    return new User({
      id: UserId.fromTrusted(raw.id), // Skip validation for trusted data
      email: Email.fromTrusted(raw.email),
      role: raw.role as UserRole,
      passwordHash: raw.passwordHash,
      displayName: raw.displayName ?? null,
      isActive: raw.isActive,
      createdAt: DateTime.from(raw.createdAt),
      updatedAt: DateTime.from(raw.updatedAt),
    });
  }
}

```

Usage:

```

// From database
const rawUser = await db.query.users.findFirst({...});
const user = User.fromSerialized(rawUser);

```

Why Factories?

Benefit	Explanation
Encapsulation	Constructor is private; creation logic is centralized
Validation	<code>create()</code> validates before instantiating
Invariants	Ensures entities always have valid state (ID + timestamps)
Two Paths	Clear distinction: new entity vs. existing entity from DB
Immutability	Entities are readonly; factories ensure proper initialization
Compile-time Safety	Can't create entity without required fields

Private Constructor Pattern

```
export class User extends BaseEntity<UserId> implements IUser {
  // Private constructor - only factories can create instances
  private constructor(data: IUser) {
    super(data);
    this.email = data.email;
    this.role = data.role;
    // ...
  }

  static create(...) { ... }
  static fromSerialized(...) { ... }
}

// This will NOT compile:
const user = new User({...}); // ❌ Error: Constructor is private
```

4. Base Entity

The foundation of all domain entities. Provides common functionality while allowing typed IDs.

File: `src/domain/shared/base.entity.ts`

Core Implementation

```

/**
 * Base interface for all domain entities.
 * Generic over the ID type so each entity can use its own typed ID
 * (e.g., DocumentId, UserId) while still satisfying the base contract.
 */
export interface IEntity<TId extends string = UUID> {
  readonly id: TId;
  readonly createdAt: DateTime;
  readonly updatedAt: DateTime;
}

export abstract class BaseEntity<TId extends string = UUID> {
  readonly id: TId
  readonly createdAt: DateTime
  readonly updatedAt: DateTime

  protected constructor(props: BaseEntityProps<TId>) {
    this.id = props.id
    this.createdAt = props.createdAt
    this.updatedAt = props.updatedAt
  }

  protected _serialize(): { id: string; createdAt: string; updatedAt: string } {
    return {
      id: this.id,
      createdAt: this.createdAt.toISOString(),
      updatedAt: this.updatedAt.toISOString(),
    }
  }

  equals(other: BaseEntity<TId>): boolean {
    return this.id === other.id
  }
}

```


Key Concepts

4.1 Generic Over ID Type

```
// Each entity can have its own typed ID
export class User extends BaseEntity<UserId> { ... }
export class Document extends BaseEntity<DocumentId> { ... }
export class AccessPolicy extends BaseEntity<AccessPolicyId> { ... }
```

This provides **type safety**: you cannot accidentally pass a DocumentId where a UserId is expected.

4.2 Readonly Fields

All entity properties are `readonly`, making entities immutable data structures:

```
readonly id: TId
readonly createdAt: DateTime
readonly updatedAt: DateTime
```

Immutability benefits: - Thread safety - Predictable behavior - Easy to reason about - Cache-friendly

4.3 Identity Equality

Two entities are equal if they have the same ID:

```
equals(other: BaseEntity<TId>): boolean {
  return this.id === other.id
}
```

This reflects the **Domain-Driven Design** principle: entities are defined by their identity, not their attributes.

4.4 Serialization Support

The `_serialize()` method converts to wire format:

```
protected _serialize(): {
  id: string;
  createdAt: string;
  updatedAt: string
} {
  return {
    id: this.id,
    createdAt: this.createdAt.toISOString(),
    updatedAt: this.updatedAt.toISOString(),
  }
}
```

Entities override this to add their own fields:

```
serialize(): SerializedUser {
  return {
    ...this._serialize(), // Include base fields
    email: Email.toString(this.email),
    role: this.role,
    passwordHash: this.passwordHash,
    displayName: this.displayName,
    isActive: this.isActive,
  };
}
```

Serialized Type

Converts domain types to primitives for JSON/API:

```
export type Serialized<T> = {
  [K in keyof T]: T[K] extends DateTime
    ? string // DateTime → ISO string
    : T[K] extends string
    ? string
    : T[K] extends Option<infer U>
    ? Serialized<U> | null // Option<T> → T | null
    : T[K] extends (infer U)[]
    ? Serialized<U>[] // Array → recursively serialized
    : T[K] extends object
    ? Serialized<T[K]> // Object → recursively serialized
    : T[K];
};
```

CreateEntity Type

Omits auto-generated fields for creation:

```
export type CreateEntity<T extends IEntity<string>> = Omit<
  T,
  'id' | 'createdAt' | 'updatedAt'
>;
```

Used by factories to ensure you can't manually set these fields:

```
static create(data: CreateEntity<IUser>): Result<User, Error>
```

5. Repository Pattern

The Repository pattern **abstracts data access**, allowing the domain to remain persistence-ignorant.

Architecture

Domain Layer:	Interface (what)
	↓
Infrastructure Layer:	Implementation (how)

Base Repository Interface

File: `src/domain/shared/base.repository.ts`

```

export type RepositoryResult<T, E = Error> = Result<T, E>;

export abstract class BaseRepository<T extends BaseEntity<string>> {
  // Required methods - every repository MUST implement these
  abstract insert(entity: T): Promise<RepositoryResult<Option<T>, AlreadyExistsError>>;
  abstract update(entity: T): Promise<RepositoryResult<Option<T>, NotFoundError>>;

  // Optional methods - repositories CAN implement these
  fetchAll?(): Promise<RepositoryResult<Option<T[]>>>;
  fetchPaginated?(options: PaginationOptions): Promise<RepositoryResult<Option<Paginated<T>>>>;
  fetchById?(id: string): Promise<RepositoryResult<Option<T>, NotFoundError>>;
  deleteById?(id: string): Promise<RepositoryResult<Option<T>, NotFoundError>>;
  fetchBy?<U extends keyof T>(prop: U, val: T[U]): Promise<RepositoryResult<Option<T>, NotFoundError>>;
  existsBy?(prop: string, val: any): Promise<RepositoryResult<boolean>>;
  deleteBy?<U extends keyof T>(prop: U, val: T[U]): Promise<RepositoryResult<Option<T>, NotFoundError>>;
}

```

Domain Repository Interface

Each aggregate defines its own repository interface extending BaseRepository:

File: `src/domain/user/user.repository.ts`

```

export interface UserRepository extends BaseRepository<User> {
  fetchByEmail(email: string): Promise<RepositoryResult<Option<User>, UserNotFoundError>>;
  fetchById(id: string): Promise<RepositoryResult<Option<User>, UserNotFoundError>>;
  existsByEmail(email: string): Promise<RepositoryResult<boolean>>;
  fetchActiveUsers(options: PaginationOptions): Promise<RepositoryResult<Paginated<User>>>>;
  fetchInactiveUsers(options: PaginationOptions): Promise<RepositoryResult<Paginated<User>>>>;
  fetchByRole(role: UserRole, options: PaginationOptions): Promise<RepositoryResult<Paginated<User>>>>;
  search(query: string, options: PaginationOptions): Promise<RepositoryResult<Paginated<User>>>>;
  register(user: User): Promise<RepositoryResult<Option<User>, EmailAlreadyTakenError | UserDomainError>>;
  delete(userId: string): Promise<RepositoryResult<Option<User>, UserNotFoundError>>;
}

```

Repository Implementation

File: `src/infra/repositories/user.repository.impl.ts`

```

@Inject() // tsyringe DI
export class UserRepositoryImpl implements UserRepository {
    constructor(private db: DrizzleDB) {}

    // Domain → Database format
    private toDbSerialized(user: User): any {
        return {
            ...user.serialize(),
            createdAt: user.createdAt.toDate(), // Convert DateTime to Date
            updatedAt: user.updatedAt.toDate(),
        };
    }

    // Database → Domain format
    private toDomain(raw: any): User {
        return User.fromSerialized(raw); // Use factory for rehydration
    }

    async insert(entity: User): Promise<RepositoryResult<Option<User>, UserAlreadyExistsError>> {
        try {
            const dbData = this.toDbSerialized(entity);
            await this.db.insert(users).values(dbData);
            return Result.Ok(Option.Some(entity));
        } catch (error) {
            return Result.Err(new UserAlreadyExistsError(entity.id.toString()));
        }
    }

    async update(entity: User): Promise<RepositoryResult<Option<User>, UserNotFoundError>> {
        try {
            const dbData = this.toDbSerialized(entity);
            const result = await this.db.update(users)
                .set(dbData)
                .where(eq(users.id, entity.id.toString()))
                .returning();

            if (result.length === 0) {
                return Result.Err(new UserNotFoundError(entity.id.toString()));
            }
            return Result.Ok(Option.Some(entity));
        } catch (error) {
            return Result.Err(new UserNotFoundError(entity.id.toString()));
        }
    }

    async fetchById(id: string): Promise<RepositoryResult<Option<User>, UserNotFoundError>> {
        try {

```

```
const raw = await this.db.query.users.findFirst({
  where: eq(users.id, id),
});
return Result.Ok(Option.fromNullable(raw).map(this.toDomain));
} catch (error) {
  return Result.Err(new UserNotFoundError(id));
}
}

// ... more methods
}
```

Why Repository Pattern?

Benefit	Explanation
Separation	Domain defines what operations are needed; Infrastructure defines how
Testability	Can mock repositories for unit tests
Flexibility	Swap PostgreSQL for MongoDB without domain changes
Query Optimization	Repository can optimize queries without affecting domain
Transaction Control	Repositories handle transactions and connections
Single Responsibility	Each aggregate has its own repository

Repository Usage Flow

```
// Application layer
class UserService {
  constructor(private userRepo: UserRepository) {}

  async createUser(data: CreateUserDto) {
    // 1. Create domain entity using factory
    const userResult = User.create({
      email: Email.create(data.email).unwrap(),
      passwordHash: await hash(data.password),
      role: UserRole.USER,
      isActive: true,
      displayName: data.displayName
    });

    if (userResult.isErr()) return userResult;
    const user = userResult.unwrap();

    // 2. Persist via repository
    const saveResult = await this.userRepo.insert(user);

    if (saveResult.isErr()) return saveResult;

    // 3. Return saved entity
    return saveResult;
  }
}
```

6. Refined Types (Branded Types)

Compile-time type safety for primitive values. Prevents mixing up different types of IDs or values.

The Problem

Without branded types, all IDs are just strings:

```
// ✗ No type safety
function getDocument(userId: string, documentId: string): Document {
  // Oops! Passed them in wrong order - compiler doesn't catch this
  return documentRepo.fetchById(userId); // Wrong!
}

getDocument(docId, userId); // Bug! But compiles fine
```

The Solution: Branded Types

File: `src/domain/utils/refined-types/user-id.type.ts`


```

import { createRefinedType } from '@carbonteq/refined-type';
import * as z from 'zod';
import { Result } from '@carbonteq/fp';

/**
 * User ID format:
 * - Normalized to lowercase UUID
 */
export const UserIdType = createRefinedType(
  'UserId',
  z.string().uuid().toLowerCase()
);

export type UserId = typeof UserIdType.$infer; // Branded string type

export const UserId = {
  /**
   * Generates a new random UserId.
   */
  init(): UserId {
    return crypto.randomUUID().toLowerCase() as UserId;
  },

  /**
   * Creates a User ID from a string value.
   * Normalizes to lowercase and validates UUID format.
   */
  create(value: string): Result<UserId, Error> {
    const normalized = value.trim().toLowerCase();
    const result = UserIdType.create(normalized);
    if (result.isErr()) {
      return Result.Err(new Error(`Invalid UserId: ${value}`));
    }
    return result;
  },

  /**
   * Creates a User ID from a trusted source (e.g., database).
   * Assumes the value is already validated and normalized.
   * Use with caution - only for values you're certain are valid.
   */
  fromTrusted(value: string): UserId {
    return value.toLowerCase() as UserId;
  },

  /**
   * Returns the string representation of the User ID.

```

```
*/
toString(id: UserId): string {
    return id;
},
};
```

Type Safety in Action

```
//  Now the compiler catches errors
function getDocument(userId: UserId, documentId: DocumentId): Document {
    // This won't compile if you pass wrong type
    return documentRepo.fetchById(documentId);
}

getDocument(userId, docId); //  Compiles
getDocument(docId, userId); //  Error: Argument of type 'DocumentId' is not assignable to
```

Available Refined Types

Type	Validation	Usage
UUID	Valid UUID format	Generic identifiers
UserId	UUID	User entity IDs
DocumentId	UUID	Document entity IDs
DocumentVersionId	UUID	Version entity IDs
AccessPolicyId	UUID	Policy entity IDs
Email	Valid email format	Email addresses
MimeType	Valid MIME type	File types
StorageKey	Valid S3 key	File storage paths
Checksum	SHA-256 hash	File integrity
WorkspaceId	UUID	Workspace identifiers

Two Paths: Validation vs. Trust

Every refined type has two creation paths:

`create()` - Validates

Use when receiving untrusted input (API requests, user input):

```
const result = UserId.create("some-id");
if (result.isErr()) {
  // Handle invalid ID
}
const userId = result.unwrap();
```

`fromTrusted()` - Skips Validation

Use when receiving data from trusted sources (database, internal services):

```
// From database - already validated
const userId = UserId.fromTrusted(databaseRow.id);
```

Why two paths? Performance. Database data is already valid; re-validating wastes CPU cycles.

7. Functional Error Handling

Instead of throwing exceptions, we use **explicit return types** with `Result<T, E>` and `Option<T>`.

The Problem with Exceptions

```
// ❌ Traditional approach - exceptions are implicit
async function findUser(id: string): Promise<User> {
  const user = await db.query.users.findFirst({ where: { id } });
  if (!user) {
    throw new UserNotFoundError(id); // Exception is hidden in the implementation
  }
  return user;
}

// Caller doesn't know this can throw
try {
  const user = await findUser(id); // Might throw - but type doesn't say so
} catch (e) {
  // Have to catch generic Error
}
```

The Solution: Result

```
// ✅ Explicit error in return type
async function findUser(id: string): Promise<Result<User, UserNotFoundError>> {
  const user = await db.query.users.findFirst({ where: { id } });
  if (!user) {
    return Result.Err(new UserNotFoundError(id)); // Explicit error
  }
  return Result.Ok(user); // Explicit success
}

// Caller knows exactly what can happen
const result = await findUser(id);

if (result.isOk()) {
  const user = result.unwrap(); // Safe - compiler knows it's Ok
} else {
  const error = result.unwrapErr(); // Type is UserNotFoundError
  // Handle specific error
}
```

Result API

```

interface Result<T, E> {
    // Check state
    isOk(): boolean;
    isErr(): boolean;

    // Extract values (only call after checking state)
    unwrap(): T;           // Returns T if Ok, throws if Err
    unwrapErr(): E;       // Returns E if Err, throws if Ok

    // Safe extraction with defaults
    unwrapOr(defaultValue: T): T;
    unwrapOrElse(fn: () => T): T;

    // Transform
    map<U>(fn: (value: T) => U): Result<U, E>;
    mapErr<F>(fn: (error: E) => F): Result<T, F>;
    flatMap<U>(fn: (value: T) => Result<U, E>): Result<U, E>;

    // Pattern matching
    match<U>(options: { ok: (value: T) => U; err: (error: E) => U }): U;
}

```

Option for Nullable Values

Represents a value that might be absent (null/undefined):

```
interface Option<T> {
    isSome(): boolean;
    isNone(): boolean;

    unwrap(): T;           // Returns T if Some, throws if None
    unwrapOr(defaultValue: T): T;
    unwrapOrElse(fn: () => T): T;

    map<U>(fn: (value: T) => U): Option<U>;
    flatMap<U>(fn: (value: T) => Option<U>): Option<U>;

    // Conversion
    toNullable(): T | null;
}

// Creating Options
Option.Some(value);      // Contains a value
Option.None;             // Represents absence
Option.fromNullable(value); // Converts null/undefined to None
```

Repository Results

Repositories combine both patterns:

```

type RepositoryResult<T, E = Error> = Result<Option<T>, E>;

// Example: fetchById returns Result<Option<User>, UserNotFoundError>
async fetchById(id: string): Promise<RepositoryResult<User, UserNotFoundError>> {
  const raw = await this.db.query.users.findFirst({ where: { id } });
  return Result.Ok(Option.fromNullable(raw).map(this.toDomain));
}

// Usage
const result = await userRepo.fetchById(id);

if (result.isOk()) {
  const maybeUser = result.unwrap();

  if (maybeUser.isSome()) {
    const user = maybeUser.unwrap();
    // Use user
  } else {
    // User not found (but no error)
  }
} else {
  const error = result.unwrapErr();
  // Database error occurred
}

```

Benefits of Functional Error Handling

Benefit	Explanation
Explicit	Return type shows what can go wrong
Type-safe	Compiler forces error handling
Composability	Chain operations with <code>map</code> , <code>flatMap</code>
No exceptions	Control flow is explicit, not hidden
Predictable	Functions always return values, never throw
Testable	Easy to assert on error cases

8. Domain Error Hierarchy

Domain errors represent **business concepts**, not technical failures.

Base Error Classes

File: `src/domain/shared/base.errors.ts`

```
/**
 * Base class for all domain errors.
 * Domain errors should NOT contain HTTP-specific concepts like status codes.
 * See multi-level error handling to understand how errors are mapped across layers.
 */
export abstract class DomainError extends Error {
  abstract readonly code: string; // Machine-readable error code

  constructor(message: string) {
    super(message);
    this.name = this.constructor.name;
    Error.captureStackTrace(this, this.constructor);
  }
}

export abstract class ValidationError extends DomainError {
  details?: Record<string, unknown>; // Field-level validation errors
}

export abstract class NotFoundError extends DomainError {}
export abstract class AlreadyExistsError extends DomainError {}
export abstract class ConflictError extends DomainError {}
export abstract class UnauthorizedError extends DomainError {}
export abstract class InvalidOperation extends DomainError {}
```

Domain-Specific Errors

Each domain module defines its own errors:

File: `src/domain/user/user.errors.ts`


```

export class UserNotFoundError extends NotFoundError {
  readonly code = "USER_NOT_FOUND";
  constructor(userId: string) {
    super(`User with ID ${userId} not found`);
  }
}

export class UserAlreadyExistsError extends AlreadyExistsError {
  readonly code = "USER_ALREADY_EXISTS";
  constructor(userId: string) {
    super(`User with ID ${userId} already exists`);
  }
}

export class EmailAlreadyTakenError extends AlreadyExistsError {
  readonly code = "EMAIL_ALREADY_TAKEN";
  constructor(email: string) {
    super(`Email ${email} is already registered`);
  }
}

export class UserValidationError extends ValidationError {
  readonly code = "USER_VALIDATION_ERROR";
  constructor(message: string, details?: Record<string, unknown>) {
    super(message);
    this.details = details;
  }
}

export class UserUnauthorizedError extends UnauthorizedError {
  readonly code = "USER_UNAUTHORIZED";
  constructor(message: string = "Unauthorized access") {
    super(message);
  }
}

```

Error Categories

Base Class	When to Use	Examples
<code>ValidationError</code>	Input fails business rules	Invalid email format, password too short
<code>NotFoundError</code>	Resource doesn't exist	User not found, Document not found
<code>AlreadyExistsError</code>	Resource already exists	Email already taken, Duplicate ID
<code>ConflictError</code>	State conflict	Concurrent modification, Version mismatch
<code>UnauthorizedError</code>	Permission denied	Access denied, Invalid credentials
<code>InvalidOperation</code>	Business rule violation	Cannot delete active user, Document already archived

Why Domain Errors?

1. **No HTTP Concepts:** Domain doesn't know about 404, 500, etc. - those are presentation concerns
2. **Ubiquitous Language:** Error names match business concepts
3. **Layer Mapping:** Presentation layer maps domain errors to appropriate HTTP responses
4. **Testability:** Can assert on specific error types in tests
5. **Internationalization:** Error codes can be mapped to localized messages

Error Mapping Example

```
// Presentation layer maps domain errors to HTTP responses
function mapToHttpResponse(error: DomainError): Response {
  switch (true) {
    case error instanceof NotFoundError:
      return new Response(404, { message: error.message });

    case error instanceof ValidationError:
      return new Response(400, {
        message: error.message,
        details: error.details
      });

    case error instanceof AlreadyExistsError:
      return new Response(409, { message: error.message });

    case error instanceof UnauthorizedError:
      return new Response(401, { message: error.message });

    default:
      return new Response(500, { message: "Internal server error" });
  }
}
```

9. Dependency Injection

We use **tsyringe** for dependency injection, enabling loose coupling and testability.

Injectable Repositories

```
import { injectable } from "tsyringe";

@injectable()
export class UserRepositoryImpl implements UserRepository {
  constructor(private db: DrizzleDB) {}
  // ...
}
```

Registering Dependencies

```
// Container setup
import { container } from "tsyringe";
import { UserRepository } from "@domain/user/user.repository";
import { UserRepositoryImpl } from "@infra/repositories/user.repository.impl";

// Bind interface to implementation
container.register<UserRepository>(UserRepository, {
  useClass: UserRepositoryImpl
});

// Or register instance
container.registerInstance("Database", db);
```

Using Dependencies

```
import { container } from "tsyringe";

class UserService {
  private userRepo = container.resolve(UserRepository);

  async createUser(data: CreateUserDto) {
    // userRepo is automatically injected with UserRepositoryImpl
  }
}
```

Constructor Injection (Preferred)

```
@injectable()
class UserController {
  constructor(
    private userService: UserService,
    private auditLogger: AuditLogger
  ) {}
}
```

Benefits of DI

Benefit	Explanation
Loose Coupling	Components depend on abstractions, not concrete classes
Testability	Easy to inject mocks for unit tests
Flexibility	Swap implementations without changing consumers
Lifecycle Management	Container manages object creation and disposal
Configuration	Wire application differently for test vs. production

10. Serialization & Rehydration

Converting between domain objects and persistence/wire formats.

Serialization Flow

```
Domain Entity → serialize() → JSON/Database Format
```

Rehydration Flow

```
Database/JSON Format → fromSerialized() → Domain Entity
```

Complete Example

```

export class User extends BaseEntity<UserId> implements IUser {
  readonly email: Email;
  readonly role: UserRole;
  readonly passwordHash: string;
  readonly displayName: string | null;
  readonly isActive: boolean;

  private constructor(data: IUser) {
    super(data);
    this.email = data.email;
    this.role = data.role;
    this.passwordHash = data.passwordHash;
    this.displayName = data.displayName;
    this.isActive = data.isActive;
  }

  // Factory: Create new entity
  static create(data: CreateEntity<IUser>): Result<User, Error> {
    const now = DateTime.now();
    return Result.Ok(new User({
      ...data,
      id: UserId.init(),
      createdAt: now,
      updatedAt: now,
    }));
  }

  // Factory: Rehydrate from persistence
  static fromSerialized(raw: SerializedUser): User {
    return new User({
      id: UserId.fromTrusted(raw.id),
      email: Email.fromTrusted(raw.email),
      role: raw.role as UserRole,
      passwordHash: raw.passwordHash,
      displayName: raw.displayName ?? null,
      isActive: raw.isActive,
      createdAt: DateTime.from(raw.createdAt),
      updatedAt: DateTime.from(raw.updatedAt),
    });
  }

  // Serialize to wire format
  serialize(): SerializedUser {
    return {
      ...this._serialize(),
      email: Email.toString(this.email),
      role: this.role,
    };
  }
}

```

```
passwordHash: this.passwordHash,  
displayName: this.displayName,  
isActive: this.isActive,  
};  
}  
}
```

Database Schema Alignment

Domain Entity:

```
{  
  id: UserId;           // Branded type  
  email: Email;         // Branded type  
  createdAt: DateTime;  // Value object  
  role: UserRole;       // Enum  
}
```

Database Model:

```
export const users = pgTable("users", {  
  id: uuid("id").primaryKey(),  
  email: varchar("email", { length: 320 }).notNull(),  
  createdAt: timestamp("createdAt", { mode: "date" }).notNull(),  
  role: userRoleEnum("role").notNull(),  
});
```

Serialized Format:

```
{  
  "id": "550e8400-e29b-41d4-a716-446655440000",  
  "email": "user@example.com",  
  "createdAt": "2024-01-15T10:30:00.000Z",  
  "role": "USER"  
}
```


Repository Serialization Methods

```

@injectable()
export class UserRepositoryImpl implements UserRepository {
  // Domain → Database (converts types)
  private toDbSerialized(user: User): any {
    return {
      ...user.serialize(),
      createdAt: user.createdAt.toDate(), // DateTime → Date
      updatedAt: user.updatedAt.toDate(),
    };
  }

  // Database → Domain (rehydrates)
  private toDomain(raw: any): User {
    return User.fromSerialized(raw);
  }

  async insert(entity: User): Promise<RepositoryResult<Option<User>, UserAlreadyExistsError>> {
    const dbData = this.toDbSerialized(entity);
    await this.db.insert(users).values(dbData);
    return Result.Ok(Option.Some(entity));
  }

  async fetchById(id: string): Promise<RepositoryResult<Option<User>, UserNotFoundError>> {
    const raw = await this.db.query.users.findFirst({ where: { id } });
    return Result.Ok(Option.fromNullable(raw).map(this.toDomain));
  }
}

```

Why Separate Serialization?

Benefit	Explanation
Type Safety	Domain uses rich types; database uses primitives
Flexibility	Can change database schema without changing domain
Validation	<code>fromSerialized</code> can validate/reject invalid data
Immutability	Serialization creates new objects; domain stays pure
Testing	Easy to create test data with serialized format

Summary: Key Architectural Patterns

Pattern	Purpose	Location
Clean Architecture	Organize code in layers with inward dependencies	Entire codebase
Factory Methods	Control object creation with validation	<code>Entity.create()</code> , <code>Entity.fromSerialized()</code>
Base Entity	Share common entity functionality	<code>src/domain/shared/base.entity.ts</code>
Repository Pattern	Abstract data access	<code>Domain: interface</code> , <code>Infra: implementation</code>
Refined Types	Type-safe identifiers and values	<code>src/domain/utils/refined-types/</code>
Result/Option	Explicit error handling	<code>@carbonteq/fp</code>
Domain Errors	Business-specific error types	<code>src/domain/shared/base.errors.ts</code>
Dependency Injection	Loose coupling and testability	tsyringe
Serialization	Convert between domain and persistence	Entity methods + Repository helpers

Technology Stack

Category	Technology
Runtime	Bun (v1.3.5+)
Language	TypeScript (ES2022)
Database	PostgreSQL
ORM	Drizzle ORM
Web Framework	Express.js
Dependency Injection	tsyringe
Functional Programming	<code>@carbonteq/fp</code> (Result/Option)
Validation	Zod + <code>@effect/schema</code>
Branded Types	<code>@carbonteq/refined-type</code>
Testing	Vitest + testcontainers

Best Practices

1. **Domain Never Imports Infrastructure** - Keep layers clean
2. **Use Factories for Creation** - Never call `new Entity()` directly
3. **Explicit Error Handling** - Use `Result<T, E>` instead of exceptions
4. **Type Safety First** - Use branded types for all IDs and constrained values
5. **Immutability** - All entities and value objects are readonly
6. **Test at Domain Layer** - Business logic tested without database/HTTP
7. **Repository Interface in Domain** - Implementation in Infrastructure
8. **Serialize at Boundaries** - Convert to primitives when leaving domain
9. **Domain Errors Are Business Concepts** - No HTTP status codes in domain

10. **Dependency Injection** - Use tsyringe for loose coupling